CURSO · HERRAMIENTAS DE DESARROLLO

Safari Kids
Informe de integración de herramientas de desarrollo

Sistema de seguimiento de incidentes, integración continua, entrega continua y plataforma en la nube aplicados al proyecto Safari Kids (Spring Boot + React + MySQL/PostgreSQL).

GitHub Issues | GitHub Actions (CI) | Railway (CD + Cloud) | Vercel (CD + Cloud)

Repositorio: github.com/kafe074/safari-kids

Fecha: 03 de julio de 2026

Autor: kafe074

0 Resumen ejecutivo

El curso pide evidenciar cuatro categorías de herramientas de desarrollo. La siguiente tabla resume qué se usó, dónde vive la configuración dentro del repositorio y cómo se verifica que funciona.

Categoría	Herramienta	Ubicación en el repo	Estado
Seguimiento de incidentes	GitHub Issues	`.github/ISSUE_TEMPLATE/`	**Implementado**
Integración continua (CI)	GitHub Actions	`.github/workflows/ci.yml`	**Implementado**
Entrega continua (CD)	Railway + Vercel (autodeploy)	`backend/railway.toml`, `frontend/vercel.json`	**Implementado**
Plataforma en la nube	Railway (backend + PostgreSQL) y Vercel (frontend)	Paneles de Railway/Vercel, conectados al repo de GitHub	**Implementado**

> **Idea central para presentar el trabajo**
>
> Las cuatro herramientas están conectadas entre sí por el mismo evento: un `git push` a `main`. Ese único push dispara en paralelo (a) el pipeline de CI en GitHub Actions, y (b) los despliegues automáticos de Railway y Vercel. El sistema de incidentes (Issues) es el punto de entrada del ciclo: de ahí sale el trabajo que termina convertido en un commit que dispara todo lo anterior.

1 Sistema de seguimiento de incidentes — GitHub Issues

Se usa **GitHub Issues**, nativo del repositorio (no requiere cuenta ni herramienta externa como Jira o Trello). Se configuraron dos plantillas para estandarizar cómo se reporta un problema o se pide una funcionalidad, y un archivo de configuración que controla el selector de plantillas.

Archivos añadidos

- `.github/ISSUE_TEMPLATE/bug_report.md` — plantilla para reportar incidentes.
- `.github/ISSUE_TEMPLATE/feature_request.md` — plantilla para pedir nuevas funcionalidades.
- `.github/ISSUE_TEMPLATE/config.yml` — habilita el selector de plantillas al crear un issue nuevo.

`.github/ISSUE_TEMPLATE/bug_report.md`

```
---
name: Reporte de bug
about: Reportar un incidente o comportamiento inesperado del sistema
title: "[BUG] "
labels: bug
---

## Descripción del incidente
## Pasos para reproducir
## Comportamiento esperado
## Comportamiento actual
## Entorno   (módulo, rol de usuario, local/producción, navegador)
## Severidad (Crítica / Alta / Media / Baja)
```

Cómo se usa en la práctica

1. Cualquier persona del equipo entra a la pestaña **Issues** del repositorio y hace clic en *New issue*.
2. GitHub muestra el selector con las dos plantillas; se elige *Reporte de bug* o *Solicitud de funcionalidad*.
3. El issue queda etiquetado automáticamente (`bug` o `enhancement`) y con un título prellenado (`[BUG] ...` / `[FEATURE] ...`).
4. El incidente se cierra referenciándolo desde el commit o PR que lo resuelve (ej. `Closes #12`), quedando trazable de incidente → código → despliegue.

2 Integración continua (CI) — GitHub Actions

Se implementó un workflow de **GitHub Actions** que corre automáticamente en cada `push` o `pull request` contra `main`. Ejecuta dos jobs en paralelo, uno por cada parte del stack.

.github/workflows/ci.yml

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  backend:
    name: Backend (Maven build + tests)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4    # JDK 17 (Temurin)
      - run: mvn -B clean verify        # build + 40 tests (H2 en memoria)

  frontend:
    name: Frontend (npm build)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4     # Node 18
      - run: npm ci
      - run: npm run build              # valida el build de Vite
```

Qué valida cada job

Job	Herramienta	Qué corre	Por qué no necesita una BD externa
`backend`	Maven + JUnit + MockMvc	`mvn clean verify` — compila y ejecuta 40 tests (unitarios + de integración de controladores)	Los tests usan el perfil `test` (`@ActiveProfiles("test")`), que apunta a H2 en memoria en vez de MySQL/PostgreSQL
`frontend`	npm + Vite	`npm ci` + `npm run build`	No aplica (no depende de BD); valida que el bundle de producción compile sin errores

> **Fix necesario para que el pipeline arrancara en verde**
>
> Al correr `mvn clean verify` por primera vez se detectó que `AuthControllerIntegrationTest.register_correoNuevo_creaClienteYRetornaToken` fallaba (esperaba 200, recibía 400). La causa: un cambio anterior del proyecto (2026-06-25) agregó la validación de que el registro exige DNI de 8 dígitos, pero el test nunca se actualizó y seguía enviando el request sin `dni`. Se corrigió agregando `req.setDni("87654321")` al test. No es un bug introducido por el CI: es deuda de tests que el

> propio proceso de configurar CI dejó en evidencia — lo cual es, en sí mismo, uno de los beneficios de tener integración continua.

Cada push queda visible en la pestaña **Actions** del repositorio con el resultado (✅ verde / ❌ rojo) de cada job, con logs completos de compilación y de cada test ejecutado.

3 Entrega continua (CD) — Railway y Vercel

El backend y el frontend ya estaban conectados directamente al repositorio de GitHub desde antes de este trabajo. Cada uno escucha los pushes a `main` y dispara su propio build + deploy, sin intervención manual.

```
git push origin main | ├─► Railway detecta el cambio → mvn clean package -DskipTests → java -jar | └─► se conecta a PostgreSQL vía SPRING_DATASOURCE_URL | └─► Vercel detecta el cambio → npm run build → deploy de dist/ └─► vercel.json reescribe /api/* hacia el backend de Railway
```

backend/railway.toml

```toml
[build]
builder = "NIXPACKS"
buildCommand = "mvn clean package -DskipTests"

[deploy]
startCommand = "java -jar target/safari-kids-backend-1.0.0.jar"
restartPolicyType = "ON_FAILURE"
restartPolicyMaxRetries = 3
```

frontend/vercel.json

```json
{
  "rewrites": [
    { "source": "/api/:path*", "destination": "https://safari-kids.up.railway.app/api/:path*"
},
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

> **Relación entre CI y CD en este proyecto**
>
> Son dos mecanismos independientes: el CI (GitHub Actions) valida el código, pero hoy **no bloquea** el deploy — Railway compila con `-DskipTests` y despliega sin esperar el resultado de Actions. Es una decisión válida para el alcance del curso (se prioriza feedback rápido de CI en PRs), pero vale mencionarlo como punto de mejora: encadenar CI → CD con un gate (deploy solo si el job de Actions termina en verde) sería el siguiente paso natural.

4 Plataforma en la nube — Railway y Vercel

Servicio	Rol	Detalle
Railway	Backend (Spring Boot) + Base de datos	Build con NIXPACKS, detecta `railway.toml` automáticamente. Incluye PostgreSQL 16 gestionado (plugin de Railway), con variables de conexión inyectadas automáticamente al servicio del backend.
Vercel	Frontend (React/Vite)	Hosting estático + edge network. Sirve el `dist/` generado por Vite y reescribe `/api/*` hacia Railway mediante `vercel.json`, evitando problemas de CORS entre dominios distintos.

URLs de producción

- Frontend: `https://safarikids.vercel.app`
- Backend API: `https://safari-kids.up.railway.app`

Ambas plataformas son PaaS (Platform-as-a-Service): no se administra el servidor ni el sistema operativo, solo el código y las variables de entorno. Esto es justamente lo que las hace "herramientas de plataforma en la nube" en el sentido del curso: abstraen infraestructura (contenedores, red, TLS, escalado) para que el equipo se concentre en la aplicación.